Task:
A Use-Case of booking a hotel room: Airbnb

Phase 2: Development

BSC in Computer Science

Instructor: Sharam Dadashnia

Date: 10th April, 2026

Submitted By: Zubair Khan

Matriculation No.: 92127983

Github URL: https://github.com/zbr-khn/SQL/tree/main

Introduction

This project implements a relational database management system for an Airbnb booking platform using MySQL. The database was designed from the Entity Relationship Diagram developed in Phase 1 and includes 21 related tables representing users, hosts, guests, bookings, reservations, payments, listings and supporting entities.

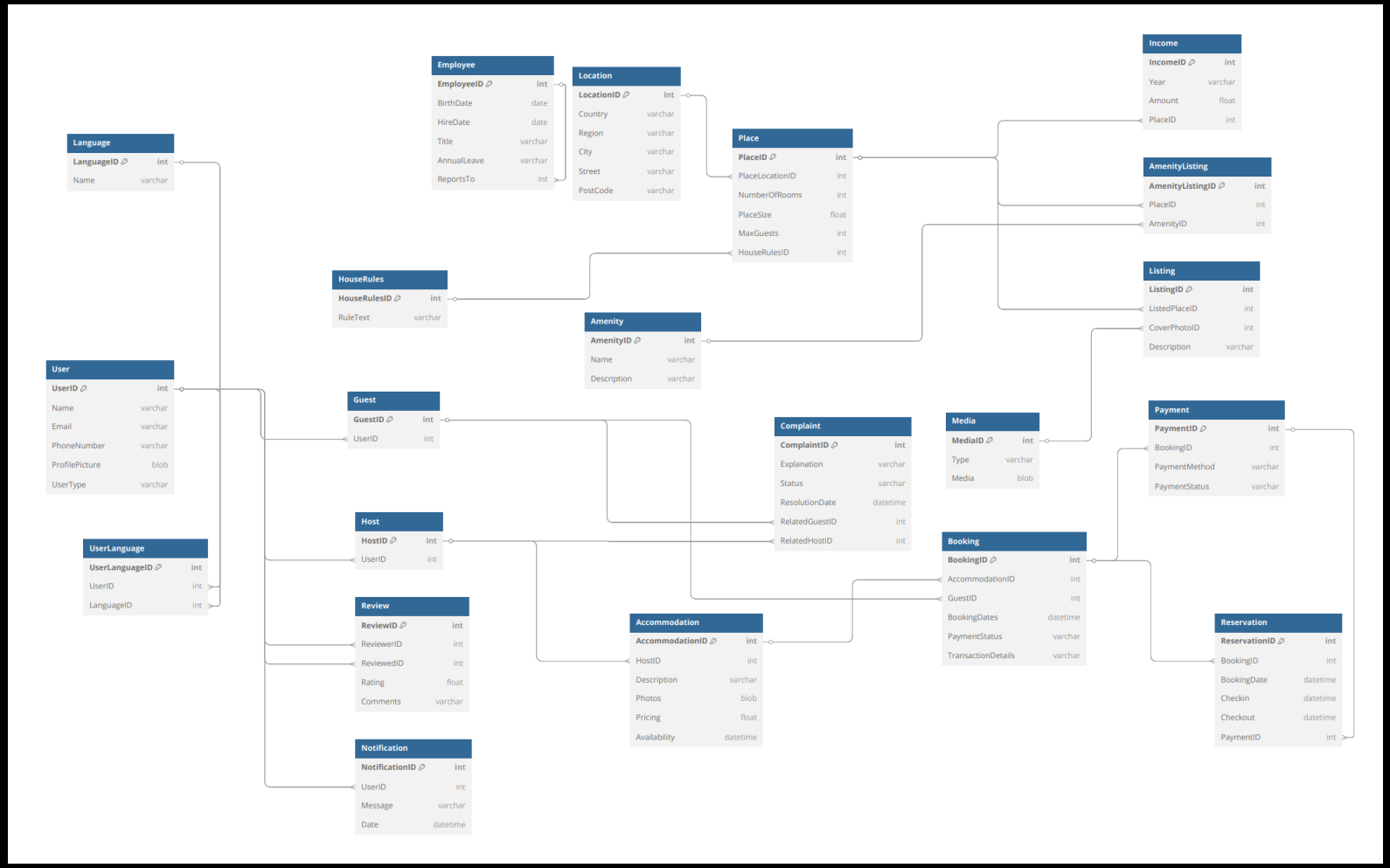
Technical highlights include:

- The database management system was developed using MySQL and is based on the Entity Relationship (ER) Model designed in Phase 1. The database consists of well-structured tables that are connected through primary and foreign key relationships, ensuring data integrity and consistency throughout the system. This relational structure allows different entities to interact efficiently while minimizing data redundancy.
- The system supports the core functionality of an Airbnb-style booking platform, including user management, property listings, bookings, reservations, payments, reviews, and notifications. Each feature is stored in its own table while remaining connected through relationships, making the database easy to manage, maintain, and expand in the future.
- By following the principles of database normalization and relational database design, the system provides accurate data storage, reliable relationships between tables, and efficient data retrieval through SQL queries. This structure ensures that the platform can handle everyday booking operations while maintaining consistency, scalability, and overall performance.

ER Diagram

At the centre lies an Entity Relationship Model - this structure guides how pieces within the system link together. Built around key elements such as Users, Guests, Hosts, and Accommodations, it manages essential functions including reservations and financial transactions.

Reality checks shape how users trust a system, so features such as guest feedback, service issues, and facility details were added into the design. Because accuracy matters when records interact, related entries connect through enforced links across tables - this means each reservation or financial record aligns only with its matching person and rental space.



User Tables

Among stored records, the User table holds core details like names, emails, and phone numbers for every individual using the system. Built into the structure of the database, this main reference point supports two distinct roles - Guest and Host - branching out from shared attributes. Through this setup, duplication drops while consistency across entries rises.
Every user gets a unique spot through UserID, acting as the main identifier. What makes one person different from another shows up clearly when UserType sorts roles apart naturally.
By cutting repeated entries, this setup keeps information more uniform while making connections to tables like Guest, Host, or Notification easier to handle. Though cleaner structure emerges, managing links becomes less tangled over time through subtle improvements in layout flow.
Foundational to the relational setup, this table links to several related entities via external key ties.

SQL Statement:

```
CREATE TABLE `user` (
  `UserID` int NOT NULL,
  `Name` varchar(100) DEFAULT NULL,
  `Email` varchar(100) DEFAULT NULL,
  `PhoneNumber` varchar(15) DEFAULT NULL,
  `ProfilePicture` blob,
  `UserType` varchar(20) DEFAULT NULL,
  PRIMARY KEY (`UserID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

SELECT * FROM User;

Result:

UserID	Name	Email	PhoneNumber	ProfilePicture	UserType
1	Michael Keith	michael@gmail.com	9999999999	NULL	Guest
2	Ali Ahmed	ali@gmail.com	8888888888	NULL	Host
3	Sara Khan	sara@gmail.com	7777777777	NULL	Guest
4	John Doe	john@gmail.com	6666666666	NULL	Host
5	User5	user5@mail.com	9000000005	NULL	Guest
6	User6	user6@mail.com	9000000006	NULL	Host
7	User7	user7@mail.com	9000000007	NULL	Guest
8	User8	user8@mail.com	9000000008	NULL	Host
9	User9	user9@mail.com	9000000009	NULL	Guest
10	User10	user10@mail.com	9000000010	NULL	Host
11	User11	user11@mail.com	9000000011	NULL	Guest
12	User12	user12@mail.com	9000000012	NULL	Host
13	User13	user13@mail.com	9000000013	NULL	Guest
14	User14	user14@mail.com	9000000014	NULL	Host
15	User15	user15@mail.com	9000000015	NULL	Guest
16	User16	user16@mail.com	9000000016	NULL	Host
17	User17	user17@mail.com	9000000017	NULL	Guest
18	User18	user18@mail.com	9000000018	NULL	Host
19	User19	user19@mail.com	9000000019	NULL	Guest
20	User20	user20@mail.com	9000000020	NULL	Host
21	User21	user21@mail.com	9000000021	NULL	Guest
22	User22	user22@mail.com	9000000022	NULL	Host
23	User23	user23@mail.com	9000000023	NULL	Guest
24	User24	user24@mail.com	9000000024	NULL	Host
NULL	NULL	NULL	NULL	NULL	NULL

Guest Table

Among those using the platform to reserve stays, some appear in the Guest table. This setup comes from the User entity, linked by UserID acting as a reference point across tables.
Because a foreign key is applied, every guest entry links back to a valid user already present. That connection stops stray data from appearing by accident. Consistency throughout the database holds firm when relationships stay anchored like this. Without such structure, mismatches could grow unchecked.
Splitting Guest from User lets the system organize data by roles yet keep structure clean. Because of this separation, tasks like managing reservations or feedback run smoothly for guests alone - without repeating core details meant for all users.
With its structure designed around user interactions, this table supports functions tied to reservations while connecting individuals to records like Booking and Review. Though unseen, it forms pathways that allow data exchange across key operational points in the system.

SQL Statement:

```
CREATE TABLE `guest` (
  `GuestID` int NOT NULL,
  `UserID` int DEFAULT NULL,
  PRIMARY KEY (`GuestID`),
  KEY `UserID` (`UserID`),
  CONSTRAINT `guest_ibfk_1` FOREIGN KEY (`UserID`) REFERENCES `user` (`UserID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

SELECT * FROM Guest;

Result:

GuestID	UserID
1	1
2	3
3	5
13	6
4	7
14	8
5	9
15	10
6	11
16	12
7	13
17	14
8	15
18	16
9	17
19	18
10	19
20	20
11	21
21	22
12	23
22	24
NULL	NULL

Host Table

The Host table represents users who provide accommodations on the platform. Similar to the Guest table, it is derived from the User entity and maintains a relationship through the foreign key (UserID).
Because of the foreign key, each host links only to an existing user within the system. Through this constraint, accuracy across related data elements remains intact. Without such alignment, mismatches could appear unexpectedly. Ensuring these connections exist reduces chances for errors over time.
One reason the structure splits Host from User lies in enabling roles without disrupting normalized design. Where management of stays and rentals falls to hosts, a distinct table becomes necessary. This connection ties individual profiles to places and listings through structured relationships.
Ownership records rely on this structure to link hosts with their listed properties. Where one entry stands, a connection forms between provider and place. Though simple in design, its role supports accurate tracking across the system. From here, assignments gain clarity without extra steps. Behind each listing sits this foundation, silent but required.

SQL Statement:

```
CREATE TABLE `host` (
  `HostID` int NOT NULL,
  `UserID` int DEFAULT NULL,
  PRIMARY KEY (`HostID`),
  KEY `UserID` (`UserID`),
  CONSTRAINT `host_ibfk_1` FOREIGN KEY (`UserID`) REFERENCES `user` (`UserID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Host;
```

Result:

HostID	UserID
15	1
1	2
13	2
16	3
2	4
14	4
17	5
3	6
18	7
4	8
19	9
5	10
20	11
6	12
21	13
7	14
22	15
8	16
9	18
10	20
11	22
12	24
NULL	NULL

Accommodation Table

One entry in the Accommodation table stands for a property offered by a host via the system. Found within this structure, every lodging links to one provider using HostID as a reference point. Connection forms when property details meet ownership records across datasets.
Essential details appear within this structure: nightly cost, availability status, descriptive notes, along with data tied to location. Preferences guide user choices during browsing tasks involving lodging options. Information flows in a way that supports selection processes without extra steps.
Because foreign keys are applied, data connections stay accurate, tying each accommodation to an existing host along with associated items like Place or Listing. With this structure in place, lookups proceed smoothly while uniformity throughout the database remains intact.
Separate storage of lodging details enables normalized design, yet still supports growth when handling many entries along with their properties. Though organized apart, the arrangement adapts smoothly as listing volumes increase together with related features.
At the center stands this table, holding essential details for property entries. From it, booking functions emerge alongside guest reviews and reservation handling. Its structure supports key processes across the platform. Without deviation, each operation ties back to its framework.

SQL Statement:

```
CREATE TABLE `accommodation` (
  `AccommodationID` int NOT NULL,
  `HostID` int DEFAULT NULL,
  `Title` varchar(100) DEFAULT NULL,
  `Location` varchar(100) DEFAULT NULL,
  `PricePerNight` decimal(10,2) DEFAULT NULL,
  `Description` varchar(255) DEFAULT NULL,
  `Photos` blob,
  `Availability` datetime DEFAULT NULL,
  `PlaceID` int DEFAULT NULL,
  PRIMARY KEY (`AccommodationID`),
  KEY `HostID` (`HostID`),
  KEY `PlaceID` (`PlaceID`),
  CONSTRAINT `accommodation_ibfk_1` FOREIGN KEY (`HostID`) REFERENCES `host` (`HostID`),
  CONSTRAINT `accommodation_ibfk_2` FOREIGN KEY (`PlaceID`) REFERENCES `place` (`PlaceID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Result:

AccommodationID	HostID	Title	Location	PricePerNight	Description	Photos	Availability	PlaceID
1	1	Sea View Apartment	Goa	5000.00	NULL	NULL	NULL	NULL
2	2	Mountain Cabin	Manali	3000.00	NULL	NULL	NULL	NULL
3	1	Stay 1	Delhi	2000.00	Nice place 1	NULL	2024-05-01 00:00:00	1
4	2	Stay 2	Delhi	2500.00	Nice place 2	NULL	2024-05-02 00:00:00	2
5	3	Stay 3	Delhi	3000.00	Nice place 3	NULL	2024-05-03 00:00:00	3
6	4	Stay 4	Delhi	3500.00	Nice place 4	NULL	2024-05-04 00:00:00	4
7	5	Stay 5	Delhi	4000.00	Nice place 5	NULL	2024-05-05 00:00:00	5
8	6	Stay 6	Bangalore	4500.00	Nice place 6	NULL	2024-05-06 00:00:00	6
9	7	Stay 7	Bangalore	5000.00	Nice place 7	NULL	2024-05-07 00:00:00	7
10	8	Stay 8	Bangalore	5500.00	Nice place 8	NULL	2024-05-08 00:00:00	8
11	9	Stay 9	Bangalore	6000.00	Nice place 9	NULL	2024-05-09 00:00:00	9
12	10	Stay 10	Bangalore	6500.00	Nice place 10	NULL	2024-05-10 00:00:00	10
13	11	Stay 11	Mumbai	7000.00	Nice place 11	NULL	2024-05-11 00:00:00	11
14	12	Stay 12	Mumbai	7500.00	Nice place 12	NULL	2024-05-12 00:00:00	12
15	13	Stay 13	Mumbai	8000.00	Nice place 13	NULL	2024-05-13 00:00:00	13
16	14	Stay 14	Mumbai	8500.00	Nice place 14	NULL	2024-05-14 00:00:00	14
17	15	Stay 15	Mumbai	9000.00	Nice place 15	NULL	2024-05-15 00:00:00	15
18	16	Stay 16	Delhi	9500.00	Nice place 16	NULL	2024-05-16 00:00:00	16
19	17	Stay 17	Delhi	10000.00	Nice place 17	NULL	2024-05-17 00:00:00	17
20	18	Stay 18	Delhi	10500.00	Nice place 18	NULL	2024-05-18 00:00:00	18
21	19	Stay 19	Delhi	11000.00	Nice place 19	NULL	2024-05-19 00:00:00	19
22	20	Stay 20	Delhi	11500.00	Nice place 20	NULL	2024-05-20 00:00:00	20
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Test Query:

```
SELECT * FROM Accommodation;
```

Booking Table

Reservations appear in the Booking table, tied to guests and their chosen stays. Through GuestID and AccommodationID, links form across data points. Payment condition alongside transaction records offer insight into monetary flow per reservation. Relationships gain structure when identifiers connect entities behind the scenes.

From time to time, this table holds essential data about stays - dates when guests arrive and leave, plus related booking records. What connects visitors to their selected accommodations is found here.

Because foreign keys are applied, data connections remain accurate by associating every reservation with existing guest and lodging entries. As a result, monitoring stays organized while creating space for steps like transaction handling or feedback entry. Though structure appears rigid, it allows fluid movement through related tasks.

This structure functions primarily as a central record for transactions, establishing the base layer of handling reservations across the platform. While essential, it operates quietly beneath higher-level processes, supporting data integrity through consistent entry tracking.

SQL Statement:

```
CREATE TABLE `booking` (
  `BookingID` int NOT NULL,
  `GuestID` int DEFAULT NULL,
  `AccommodationID` int DEFAULT NULL,
  `CheckInDate` date DEFAULT NULL,
  `CheckOutDate` date DEFAULT NULL,
  `TotalPrice` decimal(10,2) DEFAULT NULL,
  `PaymentStatus` varchar(50) DEFAULT NULL,
  `TransactionDetails` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`BookingID`),
  KEY `GuestID` (`GuestID`),
  KEY `AccommodationID` (`AccommodationID`),
  CONSTRAINT `booking_ibfk_1` FOREIGN KEY (`GuestID`) REFERENCES `guest` (`GuestID`),
  CONSTRAINT `booking_ibfk_2` FOREIGN KEY (`AccommodationID`) REFERENCES `accommodation` (`AccommodationID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Result:

BookingID	GuestID	AccommodationID	CheckInDate	CheckOutDate	TotalPrice	PaymentStatus	TransactionDetails
1	1	1	2024-04-10	2024-04-15	25000.00	NULL	NULL
2	2	2	2024-05-01	2024-05-05	12000.00	NULL	NULL
3	1	1	2024-06-01	2024-06-05	8000.00	Paid	TXN001
4	2	2	2024-06-02	2024-06-06	9000.00	Paid	TXN002
5	3	3	2024-06-03	2024-06-07	10000.00	Pending	TXN003
6	4	4	2024-06-04	2024-06-08	11000.00	Paid	TXN004
7	5	5	2024-06-05	2024-06-09	12000.00	Paid	TXN005
8	6	6	2024-06-06	2024-06-10	13000.00	Pending	TXN006
9	7	7	2024-06-07	2024-06-11	14000.00	Paid	TXN007
10	8	8	2024-06-08	2024-06-12	15000.00	Paid	TXN008
11	9	9	2024-06-09	2024-06-13	16000.00	Pending	TXN009
12	10	10	2024-06-10	2024-06-14	17000.00	Paid	TXN010
13	11	11	2024-06-11	2024-06-15	18000.00	Paid	TXN011
14	12	12	2024-06-12	2024-06-16	19000.00	Pending	TXN012
15	13	13	2024-06-13	2024-06-17	20000.00	Paid	TXN013
16	14	14	2024-06-14	2024-06-18	21000.00	Paid	TXN014
17	15	15	2024-06-15	2024-06-19	22000.00	Pending	TXN015
18	16	16	2024-06-16	2024-06-20	23000.00	Paid	TXN016
19	17	17	2024-06-17	2024-06-21	24000.00	Paid	TXN017
20	18	18	2024-06-18	2024-06-22	25000.00	Pending	TXN018
21	19	19	2024-06-19	2024-06-23	26000.00	Paid	TXN019
22	20	20	2024-06-20	2024-06-24	27000.00	Paid	TXN020
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Test Query:

```
SELECT * FROM Booking;
```

Payment Table

The Payment table holds transaction details tied to reservations. Connected via BookingID, it references entries in the Booking table. This link ensures only legitimate reservation records receive associated payments. Each financial entry traces back reliably through this relation.
This table holds information like payment method, along with its current status and the associated date, helping the system monitor money movements reliably. Payment results can be followed over time due to this structure, an aspect critical to confirming reservations accurately while maintaining stable operations.
Because foreign keys are applied, data links stay accurate across bookings and payments. As a result, transaction details remain aligned without drift over time. Financial clarity emerges naturally when record flows follow structured paths. Processing speed improves where relationships are clearly defined ahead of time.

SQL Statement:

```
CREATE TABLE `payment` (
  `PaymentID` int NOT NULL,
  `BookingID` int DEFAULT NULL,
  `PaymentMethod` varchar(50) DEFAULT NULL,
  `PaymentStatus` varchar(50) DEFAULT NULL,
  `PaymentDate` date DEFAULT NULL,
  PRIMARY KEY (`PaymentID`),
  KEY `BookingID` (`BookingID`),
  CONSTRAINT `payment_ibfk_1` FOREIGN KEY (`BookingID`) REFERENCES `booking` (`BookingID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Payment;
```

Result:

PaymentID	BookingID	PaymentMethod	PaymentStatus	PaymentDate
1	1	Credit Card	Paid	2024-06-01
2	2	PayPal	Paid	2024-06-02
3	3	Debit Card	Pending	2024-06-03
4	4	Bank Transfer	Paid	2024-06-04
5	5	Credit Card	Paid	2024-06-05
6	6	PayPal	Pending	2024-06-06
7	7	Debit Card	Paid	2024-06-07
8	8	Credit Card	Paid	2024-06-08
9	9	Bank Transfer	Pending	2024-06-09
10	10	PayPal	Paid	2024-06-10
11	11	Credit Card	Paid	2024-06-11
12	12	Bank Transfer	Pending	2024-06-12
13	13	Debit Card	Paid	2024-06-13
14	14	Credit Card	Paid	2024-06-14
15	15	Bank Transfer	Pending	2024-06-15
16	16	PayPal	Paid	2024-06-16
17	17	Credit Card	Paid	2024-06-17
18	18	Bank Transfer	Pending	2024-06-18
19	19	Debit Card	Paid	2024-06-19
20	20	Credit Card	Paid	2024-06-20
NULL	NULL	NULL	NULL	NULL

Reservation Table

A booking confirmation appears within the Reservation table, which holds supplementary data for managing stays. Connected via BookingID, this entry ties back to the primary record in the Booking table.
This table holds details like the booking date, when guests arrive and leave, also connects payments via PaymentID. With it, each reservation moves clearly from start to finish, while alignment between bookings and transactions stays consistent.
With foreign keys in place, data accuracy improves through enforced links between reservations, bookings, and payments. Because each entry ties back correctly, handling reservation records becomes more systematic.
Efficiency emerges not from added tools but from structured relationships. Organization within the system grows as dependencies remain consistent. Valid connections form the base for reliable operations.
This structure divides reservation monitoring from the process of making bookings, which increases flexibility within the system while enabling later additions like managing changes or cancellations. While separation occurs here, functionality remains clear and distinct across components.

SQL Statement:

```
CREATE TABLE `reservation` (
  `ReservationID` int NOT NULL,
  `BookingID` int DEFAULT NULL,
  `BookingDate` datetime DEFAULT NULL,
  `CheckIn` datetime DEFAULT NULL,
  `CheckOut` datetime DEFAULT NULL,
  `PaymentID` int DEFAULT NULL,
  PRIMARY KEY (`ReservationID`),
  KEY `BookingID` (`BookingID`),
  KEY `PaymentID` (`PaymentID`),
  CONSTRAINT `reservation_ibfk_1` FOREIGN KEY (`BookingID`) REFERENCES `booking` (`BookingID`),
  CONSTRAINT `reservation_ibfk_2` FOREIGN KEY (`PaymentID`) REFERENCES `payment` (`PaymentID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Reservation;
```

Result:

ReservationID	BookingID	BookingDate	CheckIn	CheckOut	PaymentID
1	1	2024-05-25 00:00:00	2024-06-01 00:00:00	2024-06-05 00:00:00	1
2	2	2024-05-26 00:00:00	2024-06-02 00:00:00	2024-06-06 00:00:00	2
3	3	2024-05-27 00:00:00	2024-06-03 00:00:00	2024-06-07 00:00:00	3
4	4	2024-05-28 00:00:00	2024-06-04 00:00:00	2024-06-08 00:00:00	4
5	5	2024-05-29 00:00:00	2024-06-05 00:00:00	2024-06-09 00:00:00	5
6	6	2024-05-30 00:00:00	2024-06-06 00:00:00	2024-06-10 00:00:00	6
7	7	2024-05-31 00:00:00	2024-06-07 00:00:00	2024-06-11 00:00:00	7
8	8	2024-06-01 00:00:00	2024-06-08 00:00:00	2024-06-12 00:00:00	8
9	9	2024-06-02 00:00:00	2024-06-09 00:00:00	2024-06-13 00:00:00	9
10	10	2024-06-03 00:00:00	2024-06-10 00:00:00	2024-06-14 00:00:00	10
11	11	2024-06-04 00:00:00	2024-06-11 00:00:00	2024-06-15 00:00:00	11
12	12	2024-06-05 00:00:00	2024-06-12 00:00:00	2024-06-16 00:00:00	12
13	13	2024-06-06 00:00:00	2024-06-13 00:00:00	2024-06-17 00:00:00	13
14	14	2024-06-07 00:00:00	2024-06-14 00:00:00	2024-06-18 00:00:00	14
15	15	2024-06-08 00:00:00	2024-06-15 00:00:00	2024-06-19 00:00:00	15
16	16	2024-06-09 00:00:00	2024-06-16 00:00:00	2024-06-20 00:00:00	16
17	17	2024-06-10 00:00:00	2024-06-17 00:00:00	2024-06-21 00:00:00	17
18	18	2024-06-11 00:00:00	2024-06-18 00:00:00	2024-06-22 00:00:00	18
19	19	2024-06-12 00:00:00	2024-06-19 00:00:00	2024-06-23 00:00:00	19
20	20	2024-06-13 00:00:00	2024-06-20 00:00:00	2024-06-24 00:00:00	20
NULL	NULL	NULL	NULL	NULL	NULL

Review Table

Found within the system, the Review table holds evaluations made by individuals about stays and exchanges between people. Connections form among participants via designated roles - one offering judgment, the other receiving it - linked through reference identifiers from user records.
This structure holds details like feedback scores alongside written reviews, which helps the platform monitor consistency while following how users respond over time.
Trust grows when guests share experiences after stays. Hosts gain clarity through guest reflections over time. Quality tends to rise where feedback becomes visible. Reputation shifts subtly with each written comment. The system holds steady mainly because voices accumulate.
Because foreign keys are applied, data connections remain accurate by assigning each review to an existing user. As a result, past interactions become reliable references for those evaluating options later.
This structure supports reliability on the system through clear visibility along with changes shaped by user input. Trust grows where actions are visible, adjustments responsive.

SQL Statement:

```
CREATE TABLE `review` (
  `ReviewID` int NOT NULL,
  `ReviewerID` int DEFAULT NULL,
  `ReviewedID` int DEFAULT NULL,
  `Rating` decimal(2,1) DEFAULT NULL,
  `Comments` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`ReviewID`),
  KEY `ReviewerID` (`ReviewerID`),
  KEY `ReviewedID` (`ReviewedID`),
  CONSTRAINT `review_ibfk_1` FOREIGN KEY (`ReviewerID`) REFERENCES `user` (`UserID`),
  CONSTRAINT `review_ibfk_2` FOREIGN KEY (`ReviewedID`) REFERENCES `user` (`UserID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Result:

ReviewID	ReviewerID	ReviewedID	Rating	Comments
1	1	2	4	Good stay
2	3	4	5	Excellent
3	5	6	3	Average
4	7	8	4	Nice
5	9	10	5	Perfect
6	11	12	2	Not great
7	13	14	4	Good
8	15	16	5	Amazing
9	17	18	3	Okay
10	19	20	4	Nice
11	2	1	5	Great host
12	4	3	4	Good
13	6	5	3	Okay
14	8	7	5	Excellent
15	10	9	4	Nice
16	12	11	2	Bad
17	14	13	4	Good
18	16	15	5	Perfect
19	18	17	3	Average
20	20	19	4	Nice
NULL	NULL	NULL	NULL	NULL

Test Query:

```
SELECT * FROM Review;
```

Complaint Table

The Complaint table is used to store any issues or concerns raised by users, whether they relate to bookings, accommodations, or interactions with other users on the platform. It keeps important information such as a description of the complaint, its current status, and the date it was resolved, making it easier to track and manage disputes.
This table is connected to both the Guest and Host through foreign keys, which helps identify who is involved in each complaint. These links also maintain data accuracy by ensuring that every complaint is tied to valid users. Overall, this structure helps keep things fair and transparent. It makes handling conflicts more organized, improves the overall user experience, and ensures that problems are properly tracked and resolved.

SQL Statement:

```
CREATE TABLE `complaint` (
  `ComplaintID` int NOT NULL,
  `Explanation` varchar(255) DEFAULT NULL,
  `Status` varchar(50) DEFAULT NULL,
  `ResolutionDate` datetime DEFAULT NULL,
  `RelatedGuestID` int DEFAULT NULL,
  `RelatedHostID` int DEFAULT NULL,
  PRIMARY KEY (`ComplaintID`),
  KEY `RelatedGuestID` (`RelatedGuestID`),
  KEY `RelatedHostID` (`RelatedHostID`),
  CONSTRAINT `complaint_ibfk_1` FOREIGN KEY (`RelatedGuestID`) REFERENCES `guest` (`GuestID`),
  CONSTRAINT `complaint_ibfk_2` FOREIGN KEY (`RelatedHostID`) REFERENCES `host` (`HostID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Complaint;
```

Result:

ComplaintID	Explanation	Status	ResolutionDate	RelatedGuestID	RelatedHostID
1	Noise issue	Resolved	2024-06-01 00:00:00	1	1
2	Cleanliness issue	Pending	NULL	2	2
3	Late check-in	Resolved	2024-06-02 00:00:00	3	3
4	Payment issue	Resolved	2024-06-03 00:00:00	4	4
5	Booking error	Pending	NULL	5	5
6	Host unavailable	Resolved	2024-06-04 00:00:00	6	6
7	Wrong listing	Resolved	2024-06-05 00:00:00	7	7
8	Security issue	Pending	NULL	8	8
9	Overcharge	Resolved	2024-06-06 00:00:00	9	9
10	No response	Resolved	2024-06-07 00:00:00	10	10
11	Noise issue	Resolved	2024-06-08 00:00:00	11	11
12	Cleanliness issue	Pending	NULL	12	12
13	Late check-in	Resolved	2024-06-09 00:00:00	13	13
14	Payment issue	Resolved	2024-06-10 00:00:00	14	14
15	Booking error	Pending	NULL	15	15
16	Host unavailable	Resolved	2024-06-11 00:00:00	16	16
17	Wrong listing	Resolved	2024-06-12 00:00:00	17	17
18	Security issue	Pending	NULL	18	18
19	Overcharge	Resolved	2024-06-13 00:00:00	19	19
20	No response	Resolved	2024-06-14 00:00:00	20	20
NULL	NULL	NULL	NULL	NULL	NULL

Notification Table

The Notification table stores all the messages and alerts that the system sends to users on the platform. These notifications can include things like booking confirmations, payment updates, or other important activity updates. Each notification is linked to a specific user through a foreign key (UserID), so the system knows exactly who should receive which message. The table typically includes details like the message content and the date it was sent, making it easy to keep track of notifications.
By using foreign keys, the system ensures that every notification is connected to a valid user, which keeps the data accurate and reliable. Overall, this table plays a key role in keeping users informed in real time, improving communication, responsiveness, and overall engagement on the platform.

SQL Statement:

```
CREATE TABLE `notification` (
  `NotificationID` int NOT NULL,
  `UserID` int DEFAULT NULL,
  `Message` varchar(255) DEFAULT NULL,
  `Date` datetime DEFAULT NULL,
  PRIMARY KEY (`NotificationID`),
  KEY `UserID` (`UserID`),
  CONSTRAINT `notification_ibfk_1` FOREIGN KEY (`UserID`) REFERENCES `user` (`UserID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Notification;
```

Result:

NotificationID	UserID	Message	Date
1	1	Booking confirmed	2024-06-01 00:00:00
2	2	Payment received	2024-06-02 00:00:00
3	3	New message	2024-06-03 00:00:00
4	4	Booking cancelled	2024-06-04 00:00:00
5	5	Reminder	2024-06-05 00:00:00
6	6	Booking confirmed	2024-06-06 00:00:00
7	7	Payment received	2024-06-07 00:00:00
8	8	New message	2024-06-08 00:00:00
9	9	Booking cancelled	2024-06-09 00:00:00
10	10	Reminder	2024-06-10 00:00:00
11	11	Booking confirmed	2024-06-11 00:00:00
12	12	Payment received	2024-06-12 00:00:00
13	13	New message	2024-06-13 00:00:00
14	14	Booking cancelled	2024-06-14 00:00:00
15	15	Reminder	2024-06-15 00:00:00
16	16	Booking confirmed	2024-06-16 00:00:00
17	17	Payment received	2024-06-17 00:00:00
18	18	New message	2024-06-18 00:00:00
19	19	Booking cancelled	2024-06-19 00:00:00
20	20	Reminder	2024-06-20 00:00:00
NULL	NULL	NULL	NULL

Location Table

The Location table keeps information about where properties are located, such as the country and region. This makes it easier to organize and manage listings based on geography.
It is connected to the Place table through foreign keys, so each property can be linked to a specific location. This connection helps the system understand exactly where a place belongs.
With this structure, users can easily search and filter properties based on location. It also makes the system more scalable and improves the overall experience when browsing accommodations.

SQL Statement:

```
CREATE TABLE `location` (
  `LocationID` int NOT NULL,
  `Country` varchar(100) DEFAULT NULL,
  `Region` varchar(100) DEFAULT NULL,
  `City` varchar(100) DEFAULT NULL,
  `Street` varchar(100) DEFAULT NULL,
  `PostCode` varchar(20) DEFAULT NULL,
  PRIMARY KEY (`LocationID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Location;
```

Result:

LocationID	Country	Region	City	Street	PostCode
1	India	North	Delhi	Street 1	110001
2	India	North	Delhi	Street 2	110002
3	India	North	Delhi	Street 3	110003
4	India	North	Delhi	Street 4	110004
5	India	North	Delhi	Street 5	110005
6	India	North	Delhi	Street 6	110006
7	India	North	Delhi	Street 7	110007
8	India	North	Delhi	Street 8	110008
9	India	North	Delhi	Street 9	110009
10	India	North	Delhi	Street 10	110010
11	India	South	Bangalore	Street 11	560001
12	India	South	Bangalore	Street 12	560002
13	India	South	Bangalore	Street 13	560003
14	India	South	Bangalore	Street 14	560004
15	India	South	Bangalore	Street 15	560005
16	India	West	Mumbai	Street 16	400001
17	India	West	Mumbai	Street 17	400002
18	India	West	Mumbai	Street 18	400003
19	India	West	Mumbai	Street 19	400004
20	India	West	Mumbai	Street 20	400005
NULL	NULL	NULL	NULL	NULL	NULL

HouseRules Table

The HouseRules table stores the rules and guidelines that hosts set for their properties. These rules help guests understand what is allowed and what is not during their stay.
It is connected to the Place table, so each property has its own set of rules linked to it. This makes it clear which rules apply to which listing.
Having a separate table for house rules keeps the data organized and flexible. It avoids repeating the same rules for multiple properties and helps maintain clear expectations between hosts and guests.

SQL Statement:

```
CREATE TABLE `houserules` (
  `HouseRulesID` int NOT NULL,
  `RuleText` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`HouseRulesID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM HouseRules;
```

Result:

HouseRulesID	RuleText
1	No smoking
2	No pets
3	No loud music
4	Check-in after 2 PM
5	Check-out before 11 AM
6	No parties
7	Keep clean
8	No outsiders
9	Respect neighbors
10	No damage
11	No smoking
12	No pets
13	No loud music
14	Check-in after 2 PM
15	Check-out before 11 AM
16	No parties
17	Keep clean
18	No outsiders
19	Respect neighbors
20	No damage
NULL	NULL

Place Table

The Place table represents all the properties available on the platform. It stores important details like the size of the property, number of rooms, and how many guests it can accommodate.
It is connected to the Location and HouseRules tables through foreign keys, so each property is linked to a specific location and has its own set of rules.
This table acts as a central part of the system for managing property information. It keeps listings organized and ensures that property data remains consistent and easy to manage as the platform grows.

SQL Statement:

```
CREATE TABLE `place` (
  `PlaceID` int NOT NULL,
  `PlaceLocationID` int DEFAULT NULL,
  `NumberOfRooms` int DEFAULT NULL,
  `PlaceSize` float DEFAULT NULL,
  `MaxGuests` int DEFAULT NULL,
  `HouseRulesID` int DEFAULT NULL,
  PRIMARY KEY (`PlaceID`),
  KEY `PlaceLocationID` (`PlaceLocationID`),
  KEY `HouseRulesID` (`HouseRulesID`),
  CONSTRAINT `place_ibfk_1` FOREIGN KEY (`PlaceLocationID`) REFERENCES `location` (`LocationID`),
  CONSTRAINT `place_ibfk_2` FOREIGN KEY (`HouseRulesID`) REFERENCES `houserules` (`HouseRulesID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Place;
```

Result:

PlaceID	PlaceLocationID	NumberOfRooms	PlaceSize	MaxGuests	HouseRulesID
1	1	2	500	3	1
2	2	3	600	4	2
3	3	1	400	2	3
4	4	2	550	3	4
5	5	3	700	5	5
6	6	2	520	3	6
7	7	1	350	2	7
8	8	2	580	4	8
9	9	3	750	5	9
10	10	2	600	4	10
11	11	1	300	2	11
12	12	2	450	3	12
13	13	3	650	5	13
14	14	2	500	4	14
15	15	1	350	2	15
16	16	2	550	3	16
17	17	3	700	5	17
18	18	2	520	4	18
19	19	1	300	2	19
20	20	2	480	3	20
NULL	NULL	NULL	NULL	NULL	NULL

Listing Table

The Listing table represents how properties are shown on the platform. It focuses on the presentation side by connecting a property to its description and media.
It includes details like the property description and cover photo, helping hosts showcase their listings in an appealing way.
This table links the Place and Media tables, making it easier to manage how properties are displayed without affecting the core property data. By separating property details from presentation, the system stays more flexible, organized, and easier to scale while also improving the overall user experience.

SQL Statement:

```
CREATE TABLE `listing` (
  `ListingID` int NOT NULL,
  `ListedPlaceID` int DEFAULT NULL,
  `CoverPhotoID` int DEFAULT NULL,
  `Description` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`ListingID`),
  KEY `ListedPlaceID` (`ListedPlaceID`),
  KEY `CoverPhotoID` (`CoverPhotoID`),
  CONSTRAINT `listing_ibfk_1` FOREIGN KEY (`ListedPlaceID`) REFERENCES `place` (`PlaceID`),
  CONSTRAINT `listing_ibfk_2` FOREIGN KEY (`CoverPhotoID`) REFERENCES `media` (`MediaID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Result:

ListingID	ListedPlaceID	CoverPhotoID	Description
1	1	1	Listing 1
2	2	2	Listing 2
3	3	3	Listing 3
4	4	4	Listing 4
5	5	5	Listing 5
6	6	6	Listing 6
7	7	7	Listing 7
8	8	8	Listing 8
9	9	9	Listing 9
10	10	10	Listing 10
11	11	11	Listing 11
12	12	12	Listing 12
13	13	13	Listing 13
14	14	14	Listing 14
15	15	15	Listing 15
16	16	16	Listing 16
17	17	17	Listing 17
18	18	18	Listing 18
19	19	19	Listing 19
20	20	20	Listing 20
NULL	NULL	NULL	NULL

Test Query:

SELECT * FROM Listing;

Media Table

The Media table stores images and other types of media related to property listings. It allows the platform to handle different formats by keeping details like the media type and the actual content.
It is connected to the Listing table, so each property can have multiple media files linked to it, such as photos or other visual content.
Keeping media in a separate table makes the system more organized and scalable. It also helps manage content more efficiently without affecting other parts of the database.

SQL Statement:

```
CREATE TABLE `media` (
  `MediaID` int NOT NULL,
  `Type` varchar(50) DEFAULT NULL,
  `Media` blob,
  PRIMARY KEY (`MediaID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

SELECT * FROM Media;

Result:

MediaID	Type	Media
1	Image	NULL
2	Image	NULL
3	Image	NULL
4	Image	NULL
5	Image	NULL
6	Image	NULL
7	Image	NULL
8	Image	NULL
9	Image	NULL
10	Image	NULL
11	Image	NULL
12	Image	NULL
13	Image	NULL
14	Image	NULL
15	Image	NULL
16	Image	NULL
17	Image	NULL
18	Image	NULL
19	Image	NULL
20	Image	NULL
NULL	NULL	NULL

Amenity Table

The Amenity table stores the different features available in properties, such as Wi-Fi, parking, or air conditioning. It helps keep these features consistent across all listings.
By organizing amenities in a separate table, the system can standardize how they are defined and used. This makes it easier for users to search and filter properties based on what they need.
It also avoids repeating the same information for multiple properties, keeping the data clean, consistent, and easier to manage.

SQL Statement:

```
CREATE TABLE `amenity` (
  `AmenityID` int NOT NULL,
  `Name` varchar(100) DEFAULT NULL,
  `Description` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`AmenityID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

SELECT * FROM Amenity;

Result:

AmenityID	Name	Description
1	WiFi	Internet access
2	AC	Air conditioning
3	TV	Television
4	Parking	Car parking
5	Pool	Swimming pool
6	Gym	Fitness center
7	Kitchen	Cooking area
8	Washer	Laundry
9	Heater	Heating
10	Balcony	Outdoor space
11	WiFi	Internet access
12	AC	Air conditioning
13	TV	Television
14	Parking	Car parking
15	Pool	Swimming pool
16	Gym	Fitness center
17	Kitchen	Cooking area
18	Washer	Laundry
19	Heater	Heating
20	Balcony	Outdoor space
NULL	NULL	NULL

AmenityListing Table

The AmenityListing table manages the relationship between properties and amenities. It connects listings with the features they offer.
This allows a single listing to have multiple amenities, and at the same time, the same amenity can be linked to multiple listings. It creates a many-to-many relationship that makes the system more flexible.
By using this structure, the platform can handle amenities efficiently without repeating data. It keeps everything organized and makes it easier to query and manage feature information across listings.

SQL Statement:

```
CREATE TABLE `amenitylisting` (
  `PlaceID` int NOT NULL,
  `AmenityID` int NOT NULL,
  PRIMARY KEY (`PlaceID`,`AmenityID`),
  KEY `AmenityID` (`AmenityID`),
  CONSTRAINT `amenitylisting_ibfk_1` FOREIGN KEY (`PlaceID`) REFERENCES `place` (`PlaceID`),
  CONSTRAINT `amenitylisting_ibfk_2` FOREIGN KEY (`AmenityID`) REFERENCES `amenity` (`AmenityID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM AmenityListing;
```

Result:

PlaceID	AmenityID
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20
NULL	NULL

Employee Table

The Employee table stores information about staff members who help manage and operate the platform. It includes details needed for internal use and administration.
It also supports a hierarchy by using a ManagerID, which allows the system to show relationships between employees, such as who reports to whom.
This structure helps keep the organization clear and well-managed, making it easier to handle internal operations and represent employee relationships within the system.

SQL Statement:

```
CREATE TABLE `employee` (
  `EmployeeID` int NOT NULL,
  `Name` varchar(100) DEFAULT NULL,
  `ManagerID` int DEFAULT NULL,
  PRIMARY KEY (`EmployeeID`),
  KEY `ManagerID` (`ManagerID`),
  CONSTRAINT `employee_ibfk_1` FOREIGN KEY (`ManagerID`) REFERENCES `employee` (`EmployeeID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Result:

EmployeeID	Name	ManagerID
1	Emp1	NULL
2	Emp2	1
3	Emp3	1
4	Emp4	2
5	Emp5	2
6	Emp6	3
7	Emp7	3
8	Emp8	4
9	Emp9	4
10	Emp10	5
11	Emp11	5
12	Emp12	6
13	Emp13	6
14	Emp14	7
15	Emp15	7
16	Emp16	8
17	Emp17	8
18	Emp18	9
19	Emp19	9
20	Emp20	10
NULL	NULL	NULL

Test Query:

SELECT * FROM Employee;

Language Table

The Language table stores the different languages supported by the platform. It allows the system to offer multiple language options for users.
By supporting different languages, users can interact with the platform in the language they are most comfortable with, which improves accessibility.
Keeping this information in a separate table makes the system more flexible and scalable, especially as more languages are added in the future.

SQL Statement:

```
CREATE TABLE `language` (
  `LanguageID` int NOT NULL,
  `Name` varchar(50) DEFAULT NULL,
  PRIMARY KEY (`LanguageID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

SELECT * FROM Language;

Result:

LanguageID	Name
1	English
2	German
3	French
4	Spanish
5	Italian
6	Dutch
7	Portuguese
8	Russian
9	Chinese
10	Japanese
11	English
12	German
13	French
14	Spanish
15	Italian
16	Dutch
17	Portuguese
18	Russian
19	Chinese
20	Japanese
NULL	NULL

UserLanguage Table

The UserLanguage table manages the relationship between users and the languages they prefer. It connects users with one or more languages they choose.
This allows a user to select multiple preferred languages, making the system more personalized and user-friendly. By using this many-to-many relationship, the platform stays flexible and avoids repeating data, while also improving how users interact with the system.

SQL Statement:

```
CREATE TABLE `userlanguage` (
  `UserID` int NOT NULL,
  `LanguageID` int NOT NULL,
  PRIMARY KEY (`UserID`,`LanguageID`),
  KEY `LanguageID` (`LanguageID`),
  CONSTRAINT `userlanguage_ibfk_1` FOREIGN KEY (`UserID`) REFERENCES `user` (`UserID`),
  CONSTRAINT `userlanguage_ibfk_2` FOREIGN KEY (`LanguageID`) REFERENCES `language` (`LanguageID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM UserLanguage;
```

Result:

UserID	LanguageID
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20
NULL	NULL

Income Table

The Income table stores the earnings that hosts receive from bookings. It keeps track of how much each host earns over time.
It is connected to the Host table through a foreign key, so every income record is linked to a valid host.
This table helps in tracking and analyzing host earnings, making it useful for financial management. It also improves transparency and provides valuable insights for the platform.

SQL Statement:

```
CREATE TABLE `income` (
  `IncomeID` int NOT NULL,
  `HostID` int DEFAULT NULL,
  `Amount` decimal(10,2) DEFAULT NULL,
  `Date` date DEFAULT NULL,
  PRIMARY KEY (`IncomeID`),
  KEY `HostID` (`HostID`),
  CONSTRAINT `income_ibfk_1` FOREIGN KEY (`HostID`) REFERENCES `host` (`HostID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Test Query:

```
SELECT * FROM Income;
```

Result:

IncomeID	HostID	Amount	Date
1	1	8000.00	2024-06-01
2	2	9000.00	2024-06-02
3	3	10000.00	2024-06-03
4	4	11000.00	2024-06-04
5	5	12000.00	2024-06-05
6	6	13000.00	2024-06-06
7	7	14000.00	2024-06-07
8	8	15000.00	2024-06-08
9	9	16000.00	2024-06-09
10	10	17000.00	2024-06-10
11	11	18000.00	2024-06-11
12	12	19000.00	2024-06-12
13	13	20000.00	2024-06-13
14	14	21000.00	2024-06-14
15	15	22000.00	2024-06-15
16	16	23000.00	2024-06-16
17	17	24000.00	2024-06-17
18	18	25000.00	2024-06-18
19	19	26000.00	2024-06-19
20	20	27000.00	2024-06-20
NULL	NULL	NULL	NULL

Test Case 1: Retrieval of complete booking information

This test case retrieves complete booking information by combining data from the **Booking**, **Guest**, **User**, **Accommodation**, and **Payment** tables. It verifies that the foreign key relationships between bookings, guests, accommodations, and payments are correctly implemented. The information extracted provides a complete overview of all reservations, including the guest name, accommodation details, booking dates, and payment status.

SQL Statement

```
SELECT
    b.BookingID,
    u.Name AS GuestName,
    a.Title AS Accommodation,
    b.CheckInDate,
    b.CheckOutDate,
    b.TotalPrice,
    p.PaymentMethod,
    p.PaymentStatus
FROM
    Booking b
        JOIN
    Guest g ON b.GuestID = g.GuestID
        JOIN
    User u ON g.UserID = u.UserID
        JOIN
    Accommodation a ON b.AccommodationID = a.AccommodationID
        JOIN
    Payment p ON b.BookingID = p.BookingID;
```

Explanation

SELECT Clause
Retrieves the booking ID, guest name, accommodation title, booking dates, total booking price, payment method, and payment status.
FROM Clause
Uses the **Booking** table as the primary source because every reservation begins with a booking.
JOIN Clauses
Joins the **Guest** table to identify the guest who made the booking.
Joins the **User** table to retrieve the guest's personal information.
Joins the **Accommodation** table to retrieve the booked property.
Joins the **Payment** table to retrieve payment information associated with the booking.
This verifies that all booking-related entities are correctly connected through foreign keys.

Result

BookingID	GuestName	Accommodation	CheckInDate	CheckOutDate	TotalPrice	PaymentMethod	PaymentStatus
1	Michael Keith	Sea View Apartment	2024-04-10	2024-04-15	25000.00	Credit Card	Paid
2	Sara Khan	Mountain Cabin	2024-05-01	2024-05-05	12000.00	PayPal	Paid
3	Michael Keith	Sea View Apartment	2024-06-01	2024-06-05	8000.00	Debit Card	Pending
4	Sara Khan	Mountain Cabin	2024-06-02	2024-06-06	9000.00	Bank Transfer	Paid
5	User5	Stay 1	2024-06-03	2024-06-07	10000.00	Credit Card	Paid
6	User7	Stay 2	2024-06-04	2024-06-08	11000.00	PayPal	Pending
7	User9	Stay 3	2024-06-05	2024-06-09	12000.00	Debit Card	Paid
8	User11	Stay 4	2024-06-06	2024-06-10	13000.00	Credit Card	Paid
9	User13	Stay 5	2024-06-07	2024-06-11	14000.00	Bank Transfer	Pending
10	User15	Stay 6	2024-06-08	2024-06-12	15000.00	PayPal	Paid
11	User17	Stay 7	2024-06-09	2024-06-13	16000.00	Credit Card	Paid
12	User19	Stay 8	2024-06-10	2024-06-14	17000.00	Bank Transfer	Pending
13	User21	Stay 9	2024-06-11	2024-06-15	18000.00	Debit Card	Paid
14	User23	Stay 10	2024-06-12	2024-06-16	19000.00	Credit Card	Paid
15	User6	Stay 11	2024-06-13	2024-06-17	20000.00	Bank Transfer	Pending
16	User8	Stay 12	2024-06-14	2024-06-18	21000.00	PayPal	Paid
17	User10	Stay 13	2024-06-15	2024-06-19	22000.00	Credit Card	Paid
18	User12	Stay 14	2024-06-16	2024-06-20	23000.00	Bank Transfer	Pending
19	User14	Stay 15	2024-06-17	2024-06-21	24000.00	Debit Card	Paid
20	User16	Stay 16	2024-06-18	2024-06-22	25000.00	Credit Card	Paid

Test Case 2: Calculation of host accommodation statistics and income

This test case retrieves information about hosts together with the accommodations they manage and their recorded income. It validates the relationships between the **Host**, **User**, **Accommodation**, and **Income** tables while demonstrating how business-related information can be extracted for reporting purposes.

SQL Statement

```
SELECT
    u.Name AS HostName,
    a.Title AS Accommodation,
    a.Location,
    a.Pricing,
    i.Amount AS Income,
    i.Date AS IncomeDate
FROM
    Host h
        JOIN
    User u ON h.UserID = u.UserID
        JOIN
    Accommodation a ON h.HostID = a.HostID
        JOIN
    Income i ON h.HostID = i.HostID;
```

Explanation

SELECT Clause
Retrieves the host name, accommodation title, accommodation location, accommodation price, income amount, and income date.
FROM Clause
Uses the **Host** table as the primary source.
JOIN Clauses
Joins the **User** table to retrieve the host's personal information.
Joins the **Accommodation** table to retrieve all accommodations managed by the host.
Joins the **Income** table to retrieve the income generated by the host.
This test case confirms that host records are correctly associated with both accommodation and income information.

Result

HostName	Accommodation	Location	Pricing	Income	IncomeDate
Ali Ahmed	Sea View Apartment	Goa	5000.00	8000.00	2024-06-01
Ali Ahmed	Stay 1	Delhi	2000.00	8000.00	2024-06-01
John Doe	Mountain Cabin	Manali	3000.00	9000.00	2024-06-02
John Doe	Stay 2	Delhi	2500.00	9000.00	2024-06-02
User6	Stay 3	Delhi	3000.00	10000.00	2024-06-03
User8	Stay 4	Delhi	3500.00	11000.00	2024-06-04
User10	Stay 5	Delhi	4000.00	12000.00	2024-06-05
User12	Stay 6	Bangalore	4500.00	13000.00	2024-06-06
User14	Stay 7	Bangalore	5000.00	14000.00	2024-06-07
User16	Stay 8	Bangalore	5500.00	15000.00	2024-06-08
User18	Stay 9	Bangalore	6000.00	16000.00	2024-06-09
User20	Stay 10	Bangalore	6500.00	17000.00	2024-06-10
User22	Stay 11	Mumbai	7000.00	18000.00	2024-06-11
User24	Stay 12	Mumbai	7500.00	19000.00	2024-06-12
Ali Ahmed	Stay 13	Mumbai	8000.00	20000.00	2024-06-13
John Doe	Stay 14	Mumbai	8500.00	21000.00	2024-06-14
Michael K...	Stay 15	Mumbai	9000.00	22000.00	2024-06-15
Sara Khan	Stay 16	Delhi	9500.00	23000.00	2024-06-16
User5	Stay 17	Delhi	10000.00	24000.00	2024-06-17
User7	Stay 18	Delhi	10500.00	25000.00	2024-06-18
User9	Stay 19	Delhi	11000.00	26000.00	2024-06-19
User11	Stay 20	Delhi	11500.00	27000.00	2024-06-20

Test Case 3: Retrieval of complete accommodation information

This test case retrieves comprehensive accommodation information by combining property details with geographical location and available amenities. It validates the relationships among the **Accommodation**, **Place**, **Location**, **AmenityListing**, and **Amenity** tables, ensuring that accommodations are properly linked to their physical locations and facilities.

SQL Statement

```
    SELECT
    a.Title AS Accommodation,
    loc.City,
    loc.Region,
    loc.Country,
    p.NumberOfRooms,
    p.MaxGuests,
    am.Name AS Amenity
FROM
    Accommodation a
        JOIN
    Place p ON a.PlaceID = p.PlaceID
        JOIN
    Location loc ON p.PlaceLocationID = loc.LocationID
        JOIN
    AmenityListing al ON p.PlaceID = al.PlaceID
        JOIN
    Amenity am ON al.AmenityID = am.AmenityID
ORDER BY a.Title;
```

Explanation

SELECT Clause
Retrieves accommodation title, city, region, country, number of rooms, maximum guest capacity, and available amenities.
FROM Clause
Uses the **Accommodation** table as the starting point.
JOIN Clauses
Joins the **Place** table to obtain property specifications.
Joins the **Location** table to retrieve geographical information.
Joins the **AmenityListing** table to connect properties with amenities.
Joins the **Amenity** table to retrieve the names of the available facilities.
This test case verifies that accommodation records are correctly linked with their locations and amenities.

Result

Accommodation	City	Region	Country	NumberOfRooms	MaxGuests	Amenity
Stay 1	Delhi	North	India	2	3	WiFi
Stay 10	Delhi	North	India	2	4	Balcony
Stay 11	Bangalore	South	India	1	2	WiFi
Stay 12	Bangalore	South	India	2	3	AC
Stay 13	Bangalore	South	India	3	5	TV
Stay 14	Bangalore	South	India	2	4	Parking
Stay 15	Bangalore	South	India	1	2	Pool
Stay 16	Mumbai	West	India	2	3	Gym
Stay 17	Mumbai	West	India	3	5	Kitchen
Stay 18	Mumbai	West	India	2	4	Washer
Stay 19	Mumbai	West	India	1	2	Heater
Stay 2	Delhi	North	India	3	4	AC
Stay 20	Mumbai	West	India	2	3	Balcony
Stay 3	Delhi	North	India	1	2	TV
Stay 4	Delhi	North	India	2	3	Parking
Stay 5	Delhi	North	India	3	5	Pool
Stay 6	Delhi	North	India	2	3	Gym
Stay 7	Delhi	North	India	1	2	Kitchen
Stay 8	Delhi	North	India	2	4	Washer
Stay 9	Delhi	North	India	3	5	Heater

Summary

This project presents a structured approach to designing a relational database for an online lodging platform. It is organized around key components such as guests, bookings, property details, reviews, complaints, and financial transactions. Instead of mixing everything together, the data is divided into clear, focused tables, each handling a specific purpose. This naturally creates consistency and keeps the system organized as it grows.

Each record is uniquely identified using a primary key, while relationships between tables are maintained through foreign keys. This setup keeps the data accurate and connected, making it easier to retrieve information and run complex queries across different parts of the system. Core tables like Place, Listing, and Location form the foundation, while others like Review, Complaint, and Notification add additional functionality without complicating the main structure.

Normalization plays an important role by reducing data duplication and improving efficiency. When relationships become more complex, such as linking amenities to properties, bridge tables are used to manage those connections cleanly. The database design also reflects real-world usage, allowing bookings, reviews, and earnings to be recorded in a natural and flexible way.

Scalability is built into the system from the beginning. The design supports smooth performance even as the platform grows, while keeping maintenance simple through a modular structure. Data remains reliable due to built-in constraints, and users can interact with the system without confusion or delays. Overall, each part of the database works together to create a stable, efficient, and user-friendly platform.